Gate 02 Reconciliation: Data Layer

Field	Value
Gate	02 — Data Layer
Spec version	2026-04-09 (post-conservation-map amendment)
Implementation date	2026-04-16
Reconciled by	Claude Code
Branch	gate/02-data-layer

Summary

Gate 02 was implemented as specified with four documented deviations and several minor additions. All 13 models across 5 Django apps were created, all acceptance criteria are satisfied (some via adapted mechanisms), and migrations apply cleanly with the circular dependency between accounts and populations resolved via a split migration strategy. Tests cover every acceptance criterion in the spec.

Acceptance Criteria Status

#	Criterion (from spec)	Status	Notes
1	Undescribed species with null authority/year_described saves without error	Pass	Tested in `TestSpecies.test_undescribed_species_null_aut`
2	Described species with provisional_name saves without error	Pass	Tested in `TestSpecies.test_described_species_with_provisional_nam`
3	ConservationAssessment with pending_review and null flagged_by/flagged_date saves	Pass	Tested in `TestConservationAssessment.test_pending_review_with_nul`
4	access_tier outside 1-5 raises validation error	Pass	Tested at both validator level (`full_clean`) and DB CheckConstraint level
5	Duplicate ExSituPopulation (species, institution) raises unique constraint violation	Pass	Tested in `TestExSituPopulation.test_unique_species_inst`

6	SpeciesLocality PointField stores coordinates correctly; spatial query works	Pass	Tested in `test_point_field_stores_coordinates` and `test_st_contains_spatial_query`
7	Duplicate SpeciesLocality (species, location, locality_type) raises unique constraint violation	Pass — with deviation	Uses `location_key` CharField instead of raw geometry. Se Deviations.
8	Sensitive SpeciesLocality auto-computes location_generalized as rounded to 0.1 degree	Pass	Tested in `test_sensitive_location_generalized`
9	Non-sensitive SpeciesLocality has null location_generalized	Pass	Tested in `test_non_sensitive_location_generalized_null`
10	Watershed with valid MultiPolygon and hybas_id saves; duplicate hybas_id fails	Pass	Tested in `TestWatershed`
11	ProtectedArea with wdpa_id saves; duplicate wdpa_id fails	Pass	Tested in `TestProtectedArea`

User Story Status

The spec does not define numbered user stories; it defines models, technical tasks, and acceptance criteria. All technical tasks are addressed:

Task	Status	Notes
`startapp` for species, populations, fieldwork, accounts, integration	Complete	All 5 apps created
Add apps to INSTALLED_APPS	Complete	Verified by migrations running
Custom User model before first migration	Complete	AUTH_USER_MODEL = 'accounts.User'
Enable PostGIS (django.contrib.gis, postgis backend)	Complete	Spatial fields work in tests
Install django-mptt for Taxon hierarchy	Complete	Taxon uses MPTTModel + TreeForeignKey

Run initial migrations	Complete	0001_initial for all 5 apps + 0002 for accounts
Model-level unit tests	Complete	21 tests covering all acceptance criteria

Deviations

1. SpeciesLocality unique constraint uses location_key instead of raw geometry equality

- **Spec said:** `unique_together = (species, location, locality_type)` with the PointField directly in the constraint.
- **Implementation does:** Adds a `location_key` CharField(50, editable=False) computed as `"{x:.5f},{y:.5f}"` on save. UniqueConstraint uses `fields=["species", "location_key", "locality_type"]`.
- **Reason:** PostgreSQL geometry equality is unreliable for unique constraints. Floating-point representation of coordinates means two Points with the same logical coordinates may not compare as equal at the database level. A deterministic string key provides reliable uniqueness.
- **Impact:** None for users. The constraint enforces the same semantic guarantee. The `location_key` field is non-editable and auto-computed.

2. CheckConstraint on User.access_tier uses `condition=` parameter instead of `check=`

- **Spec said:** Enforce access_tier range 1-5 with `validators=[MinValueValidator(1), MaxValueValidator(5)]`.
- **Implementation does:** Both validators AND a DB-level CheckConstraint using `condition=` parameter (Django 6.0+ forward-compatible syntax instead of deprecated `check=` parameter).
- **Reason:** Django 5.1 deprecated the `check=` parameter on CheckConstraint in favor of `condition=`. Using `condition=` avoids deprecation warnings and is forward-compatible with Django 6.0.
- **Impact:** None. Functionally identical. Provides stronger enforcement (DB-level + validator-level).

3. Circular migration dependency resolved via split migration

- **Spec said:** No specific migration strategy prescribed.
- **Implementation does:** accounts/0001_initial creates User and AuditLog without cross-app ForeignKeys. accounts/0002 adds User.institution FK (to populations.Institution) and AuditLog.user FK (to accounts.User) after populations/0001_initial exists.
- **Reason:** accounts.User references populations.Institution, while populations.HoldingRecord and fieldwork models reference accounts.User via settings.AUTH_USER_MODEL. A single 0001 migration for accounts that included the institution FK would create a circular dependency with populations.
- **Impact:** None. Migrations apply cleanly in sequence. The split is transparent to application code.

4. drainage_basin_name always syncs from FK on save

- **Spec said:** "if `drainage_basin` FK is set and `drainage_basin_name` is empty, populate from `drainage_basin.name`."
- **Implementation does:** Always syncs `drainage_basin_name` from the FK when the FK is set, regardless of whether `drainage_basin_name` is already populated.
- **Reason:** Denormalized fields that only populate when empty can silently become stale if the FK target's name changes or if the record is re-saved after a basin reassignment. Always syncing

ensures consistency.

- **Impact:** Minor behavioral difference: manually overriding `drainage_basin_name` while a FK is set will be overwritten on save. This is the safer default for data integrity. If manual overrides are needed in the future, the behavior can be adjusted.

5. SyncJob.error_log is JSONField instead of TextField

- **Spec said:** `error_log` — TextField, blank. Accumulated errors.
- **Implementation does:** `error_log = models.JSONField(default=list, blank=True)` — stores errors as a JSON list.
- **Reason:** Structured error storage (JSON list) is more useful for programmatic access, filtering, and display than a plain text blob. Each error entry can include timestamps, error codes, and affected record IDs.
- **Impact:** Functionally superior. API consumers and admin views can iterate over structured error entries rather than parsing text.

6. FieldProgram includes end_date field not in spec

- **Spec said:** FieldProgram has `start_date` but no `end_date`.
- **Implementation does:** Adds `end_date = models.DateField(null=True, blank=True)`.
- **Reason:** A field program with a `status` of "completed" but no `end_date` loses temporal information. This is a natural companion to `start_date`.
- **Impact:** None. Nullable field, fully backward-compatible.

Additions (not in spec)

QuerySet managers with `for_tier()` methods

Three custom QuerySets were added as forward-looking infrastructure for Gate 03 (Auth & Access Control):

- `SpeciesQuerySet.for_tier(tier)` — returns all species (public at all tiers).
- `ConservationAssessmentQuerySet.for_tier(tier)` — Tier 3+ sees all; below Tier 3 sees only `review_status="accepted"`.
- `ExSituPopulationQuerySet.for_tier(tier)` — Tier 3+ sees all; below Tier 3 sees nothing.

These are additive and do not affect Gate 02 behavior. They will be consumed by DRF views in Gate 05.

Institution.created_at, ExSituPopulation.created_at, HoldingRecord.created_at

Timestamp fields added to populations models that were not specified. Standard audit trail practice.

HoldingRecord ordering by `-date`

Default ordering added for convenience. Non-breaking.

Model `__str__` methods

All models have `__str__` implementations for admin and debugging readability. Not specified but standard Django practice.

DB table names explicitly set via `Meta.db_table`

All models specify explicit `db_table` names. Not required but provides stability if app labels change.

Deferred Items

None. All models and technical tasks specified in Gate 02 were implemented.

Technical Decisions Made During Implementation

1. **Django 5.1.15** — the project uses Django 5.1.x (not 5.0 or 6.0), as evidenced by the migration files. The `condition=` parameter on CheckConstraint is supported and forward-compatible.

2. **UniqueConstraint instead of unique_together** — ExSituPopulation and SpeciesLocality use `models.UniqueConstraint` (the modern Django API) rather than the deprecated `unique_together` Meta option. Functionally identical but aligned with Django's recommended approach.

3. **TextChoices enums on models** — All enum fields use Django's `models.TextChoices` classes defined within the model class, providing type safety and centralized validation.

4. **SpeciesLocality.location_key precision** — Coordinates are stored with 5 decimal places (~1.1 meter precision), which is more than sufficient for biodiversity locality data and avoids false duplicates from floating-point noise.

5. **AuditLog append-only enforcement** — Implemented via a `pre_delete` signal in `accounts/signals.py` that raises `RuntimeError`. This blocks deletion through the ORM. Direct SQL DELETE would bypass this; full protection would require a database-level trigger or row-level security (out of scope for Gate 02).

Spec Updates Needed

1. **SpeciesLocality unique constraint** — Update spec to document the `location_key` approach: `unique_together` should read `(species, location_key, locality_type)` with a note explaining the deterministic key pattern.

2. **drainage_basin_name sync behavior** — Update spec from "if drainage_basin_name is empty" to "always syncs from FK on save" to match implementation.

3. **SyncJob.error_log type** — Update spec from `TextField, blank` to `JSONField(default=list)`.

4. **FieldProgram.end_date** — Add `end_date: DateField, nullable` to the spec.

5. **CheckConstraint syntax** — Note in spec that `condition=` is used instead of `check=` for Django 5.1+ compatibility.

6. **Migration strategy** — Add a note documenting the accounts 0001/0002 split migration approach for future reference.